TP2: Filtrage d'images et apprentissage profond

GBM8770 – Automne 2026

Enseignante: Sandrine Bédard
Chargé de laboratoire: Zacharie Legault

Dernière mise à jour : 2026-10-09

Objectifs: Ce laboratoire se compose de deux parties. La première partie porte sur le filtrage spectral des images et la deuxième partie sur l'utilisation de l'apprentissage profond sur des images médicales.

Remise du travail: La date de remise est le 6 novembre à 23h30. Une pénalité de 3 points par jour sera appliquée lors d'un retard.

Documents à remettre: Le code et les réponses aux questions sont à compléter dans les notebooks IPython fournis avec cet énoncé `tp2_seance1.ipynb` et `tp2_seance2.ipynb`).

Le dossier contenant le notebook et toutes les ressources nécessaires à son exécution sont à remettre sous la forme d'une archive ZIP nommée

`GBM8770_TP2_<equipe>_<nom>_<matricule>_<nom>_<matricule>`

(par exemple: `GBM8770_TP2_0_Venne_1234567_Legault_7654321`).

Commentez votre code ! Lorsque votre code (et donc vos résultats) est incorrect, vos commentaires peuvent nous permettre de valoriser votre approche…

Séance 2: Apprentissage profond appliqué aux images médicales

Les prochains exercices vous guideront au travers des bases de l'entraînement de réseaux de neurones. Pour ce faire, et pour vous faciliter la tâche, vous devrez installer les librairies qui contiennent les modèles et les fonctions de traitement appropriées. Veuillez suivre les indications suivantes :

> ⚠ Attention ! Les réseaux de neurones peuvent facilement consommer beaucoup de ressources de votre ordinateur. Si votre ordinateur n'est pas muni d'un GPU dédié (idéalement de marque Nvidia), nous vous déconseillons de faire le TP sur votre propre ordinateur. Les ordinateurs du local L-4818 sont munis de GPU appropriés, il serait mieux de les utiliser dans ce cas. Si vous souhaitez travailler à distance sur les ordinateurs du labo en dehors de la séance de TP, vous pouvez suivre les instructions du service informatique pour vous y connecter.
>
> Sinon, vous pouvez exécuter le notebook sur Google Colab, où vous aurez accès gratuitement à un GPU. Soyez cependant conscients que Colab ne vous permet que quelques heures d'utilisation avant que vous perdiez l'accès à un GPU. Aussi, si vous perdez votre connexion Internet, l'environnement se déconnectera et vous perdrez vos variables.
>
> Si vous utilisez les ordinateurs du labo, suivez les instructions suivantes. Sinon, l'environnement Google Colab devrait déjà avoir les paquets installés.

Vous devrez commencer par installer des nouveaux modules, en spécifiant les bonnes versions.

```
$ conda activate gbm8770
$ conda install scikit-learn -y
$ pip install torch torchvision --index-url https://download.pytorch.org/whl/
cu132
$ pip install datasets
```

> ℹ Les commandes fournies ici assument que vous utilisez un ordinateur du L-4818 et que vous avez préalablement créé un environnement `gbm8770` en suivant les instructions du TP0. Si vous utilisez votre propre ordinateur, vous devrez peut-être adapter les commandes d'installation en fonction de votre configuration. Dans ce cas, référez-vous à la page d'installation de PyTorch pour obtenir les instructions spécifiques à votre système d'exploitation et à votre configuration matérielle.

> ⚠ Assurez-vous que vous pouvez exécuter la première cellule du notebook (qui importe les différentes librairies) afin de vérifier que l'installation s'est déroulée

correctement. Demandez de l'aide à vos chargés de laboratoire si l'exécution contient des erreurs.

Instructions pour l'utilisation du notebook via Google Colab :

1. Ouvrez Google Colab . Vous aurez besoin d'un compte Google.
2. Cliquez sur Open Colab.
3. Une fenêtre s'ouvrira. Sélectionnez Importer, puis venez téléverser le notebook `tp2_seance2_colab.ipynb`.
4. En haut à droite, cliquez sur Connecter, puis sur la flèche vers le bas qui viendra d'apparaître, et enfin sur Modifier le type d'exécution. Choisissez l'option T4 GPU, puis Enregistrer. Vous devriez maintenant être connectés à un environnement avec GPU.
5. Exécutez la première cellule pour installer les librairies nécessaires au TP.
6. Cliquez sur l'icône "Files" dans le panneau à gauche de l'écran pour accéder aux fichiers disponibles dans le notebook Colab. Cliquez ensuite sur l'icône "Upload to session storage" pour téléverser le fichier `utils.py` qui vous est fourni.
7. Exécutez la deuxième cellule (qui importe les différentes librairies). Si tout fonctionne, vous pouvez maintenant faire le TP ! Si vous avez des problèmes, demandez de l'aide à vos chargés.

Attention ! Dans Colab le contenu du notebook comme tel (code dans les cellules, figures générées) sera sauvegardé automatiquement. Par contre, tous les fichiers seront perdus entre deux sessions. Vous devrez téléverser à chaque nouvelle session le fichier `utils.py`. Si vous avez sauvegardé un fichier (e.g. les poids d'un modèle), pensez à le télécharger localement sur votre ordinateur, puis le téléverser dans Colab pour pouvoir le réutiliser plus tard.

Le deep learning a révolutionné de nombreux domaines, et l'imagerie médicale ne fait pas exception. Aujourd'hui, les réseaux de neurones convolutifs (CNN) sont devenus omniprésents dans la détection, la classification et la segmentation des images médicales, offrant des performances inégalées dans l'analyse des données complexes. Dans ce TP, vous serez amenés à appliquer les concepts du deep learning à un dataset d'images médicales [1], [2] constitué d'images OCT de rétines atteintes de différentes pathologies. Le dataset comprend des images OCT de sujets sains, ainsi que des exemples de néovascularisation choroïdale, où de nouveaux vaisseaux sanguins anormaux se développent sous la rétine, des exemples d'œdème maculaire diabétique, caractérisés par une accumulation de liquide au niveau de la macula souvent liée au diabète, ainsi que des exemples de drusen correspondant à des dépôts sous la rétine, fréquemment associés à la dégénérescence maculaire liée à l'âge.

Exercice 2.1: Exploration des données

Comme l'apprentissage profond utilise des jeux de données volumineux, il est important de bien comprendre la structure et la composition du dataset avant de commencer

l'entraînement d'un modèle. Comprendre les données vous aidera à choisir les bonnes architectures de modèles, les techniques de prétraitement, et les stratégies d'augmentation des données. Après l'entraînement, une bonne compréhension des données vous aidera à interpréter les résultats et à identifier les éventuels biais ou limitations du modèle.

Q1. Combien d'images y a-t-il dans chaque sous-ensemble du dataset?

> ⚠ Attention ! Si vous essayez de charger toutes les images d'un coup, ça ne marchera probablement pas car il y en a beaucoup trop pour la quantité de RAM qui est vraisemblablement disponible sur votre machine.

Q2. Quelles sont les classes représentées dans le dataset? Comment sont-elles représentées?

> ℹ La fonction `np.unique` pourrait vous être utile.

Q3. Quel nombre et proportion des images de chaque sous-ensemble appartient aux différentes classes? Affichez un histogramme de la distribution des classes pour chaque sous-ensemble.

> ℹ Vous pouvez utiliser la fonction `np.bincount` pour compter le nombre d'occurence de chaque classe.

Q4. Affichez 2 échantillons de chaque classe. Pour chaque image, affichez l'indice et la classe dans le titre. Décrivez les différences que vous observez entre les classes.

Q5. Vous remarquerez que les images ne sont pas toutes de la même taille. Identifiez les tailles d'images et leur fréquence dans chaque sous-ensemble. Que pouvez-vous observer sur leur distribution? En quoi est-ce que des images de tailles différentes peut être un problème?

> ℹ Le code que nous vous fournissons gère le problème des tailles différentes en redimensionnant les images à une taille fixe pré-déterminée. Vous pouvez aller voir le code de `get_dataloaders` et `_Transform` dans le fichier `utils.py`

Exercice 2.2: Entraînement d'un modèle de classification

> ℹ Une librairie très souvent utilisée dans l'écosystème Python est `tqdm` . Elle permet de facilement créer des barres de progression pour le suivi d'une tâche itérative.
>
> Pour l'utiliser, il suffit d'encapsuler un objet itérable (e.g. un `range`, une liste, un dataloader, etc.) avec la syntaxe `for ... in tqdm(iterable): ...`.

Les fonctions fournies en font déjà usage pour vous donner un feedback visuel lors de l'entraînement. N'hésitez pas à vous en servir pour votre propre code pour faciliter le suivi de vos expériences (e.g. `for i in tqdm(range(num_epochs)): ...`).

Le fichier `utils.py` fourni en plus du Notebook contient plusieurs fonctions pour vous faciliter la tâche pour ce TP. Prenez bien le temps de lire la documentation de chacune d'elles. Voici un résumé des fonctions que vous utiliserez:

`get_model_and_optimizer` Crée un modèle et son optimiseur. Cette fonction prend 1 argument: un booléen optionnel `pretrained` (pour indiquer si le modèle a été pré-entraîné sur l'ensemble ImageNet ou s'il a été initialisé aléatoirement).

`get_dataloaders` Crée trois dataloaders (qui permettent d'itérer sur un dataset et qui devront être passées à `train_epoch` et `validate`) – un pour chaque sous-ensemble (entraînement, validation, test). Comme le dataset qu'on utilise pour ce TP ne contient pas par défaut de sous-ensemble de validation, on en crée un en utilisant une partie des données d'entraînement. Cette fonction attend deux argument obligatoires: un `DatasetDict` (le dataset qu'on a créé plus haut) et un entier `batch_size` (qui détermine combien d'exemples le modèle traitera à chaque itération). Quelques notes additionnelles sur cette fonction:

1. En général, il est préférable d'utiliser la plus grande *batch size* possible qui entre en mémoire sur votre GPU. Vous pouvez expérimenter avec différentes valeurs (typiquement par puissances de 2) jusqu'à ce que vous trouviez une valeur qui fonctionne, mais une *batch size* de 128 devrait être suffisante et entrer en mémoire de la plupart des GPU.
2. La fonction offre un paramètre optionnel `train_subset` qui est un float compris entre 0 et 1 qui vous permet de réduire la taille de l'ensemble d'entraînement.

`train_epoch` Entraîne le modèle sur une époque, soit un passage complet de tout le jeu d'entrainement. Elle prend 3 arguments: le modèle, le dataloader d'entraînement, et l'optimiseur. Elle retourne la liste des valeurs de *loss* à chaque itération pour cette époque (i.e. par batch).

`validate` Évalue le modèle sur un ensemble de données. Elle prend 3 arguments: le modèle, un dataloader, et un booléen optionnel. Elle retourne deux floats et optionnellement une liste d'entiers: la valeur de *loss*, la précision (*accuracy*), et les prédictions du modèle sur l'ensemble qu'on valide. À noter que malgré ce que son nom pourrait laisser entendre, cette fonction peut techniquement être utilisée avec n'importe quel dataloader, pas nécessairement celui de validation. Cela pourrait être utile pour évaluer le modèle sur l'ensemble d'entraînement ou l'ensemble de test…

`get_embeddings` Extrait les vecteurs latents (embeddings) d'un ensemble de données. Elle prend 2 arguments: le modèle et un dataloader. Elle retourne un tenseur PyTorch contenant les vecteurs latents pour chaque image dans le dataloader.

Une bonne pratique avant de commencer à entraîner un modèle est de vérifier que la *loss* qu'on obtient de ce modèle aléatoirement initialisé est bel et bien ce qu'on attend. Pour un problème de classification où on a K classe et qu'on utilise la fonction d'entropie croisée (*cross-entropy*), un classificateur aléatoire devrait en moyenne produire une *loss* de $\log(K)$.

$$\ell = -\sum_{k=1}^{K} q_k \log(p_k) = -\sum_{k=1}^{K} q_k \log\left(\frac{1}{K}\right) = -\log\left(\frac{1}{K}\right) \underbrace{\sum_{k=1}^{K} q_k}_{=1} = \log(K)$$

où $p \in \mathbb{R}^K$ est la distribution multinomiale prédite par le modèle (uniforme si le modèle est aléatoire) et $q_k = 1$ si la classe est bien k et 0 sinon.

Q1. Créez un modèle qui n'a pas été pré-entraîné (utilisez la valeur par défaut de `pretrained=False`). Calculez la *loss* initiale du modèle sur 10% des données d'entraînement. Est-ce que la *loss* initiale est bien ce à quoi vous vous attendiez?

Q2. Entraînez votre modèle sur 10% des données d'entraînement[1] (20 époques devraient suffire). Affichez la courbe d'évolution de votre erreur (*loss*) d'entraînement. Au long de votre entraînement, validez la performance de votre modèle sur l'ensemble de validation et affichez la *loss* et l'*accuracy* en fonction de l'époque.

Faites le suivi de l'*accuracy* et identifiez la plus haute valeur obtenue au cours de l'entraînement. Faites une sauvegarde du modèle lorsque vous obtenez cette meilleure *accuracy*.

N.B.: Pour cette question, vous n'avez pas besoin de faire une validation à chaque époque. Vous pouvez choisir un intervalle plus grand (e.g. toutes les 2 époques) pour diminuer le temps de calcul.

Pour sauvegarder un modèle, vous pouvez utiliser la syntaxe suivante :

```
>>> torch.save(model.state_dict(), "mon_modèle.pt")
```

La méthode `state_dict()` d'un modèle PyTorch retourne un `dict` qui contient les poids du modèle, tandis que la fonction `torch.save` permet de sauvegarder un tenseur (ou une collection de tenseurs comme un `dict`) sous un nom de fichier donné.

[1] En temps normal, on voudrait utiliser le maximum de données disponible. Pour les fins du TP présent, on se permettra d'utiliser moins de données afin de diminuer le temps de calcul. Pour itérer rapidement pendant que vous explorez le notebook, vous pouvez utiliser une proportion encore plus petite (mais veillez à utiliser au moins 10% des données pour le rendu final). Si vous avez le temps, nous vous encourageons à utiliser autant de données que possible!

Pour charger un modèle à partir d'une sauvegarde, il faut d'abord créer un modèle identique, charger le `dict` des poids depuis le fichier, puis charger le `dict` de poids dans le modèle.

```
>>> saved_model = get_model_and_optimizer(pretrained=False)[0]
>>> saved_state_dict = torch.load("mon_modèle.pt", map_location="cpu")
>>> saved_model.load_state_dict(saved_state_dict)
```

Si vous travaillez sur Google Colab, pensez à télécharger vos modèles sauvegardés, parce qu'ils ne persisteront pas d'une session à l'autre. Vous pourrez les téléverser pour les réutiliser.

Exercice 2.3: Apprentissage par transfert

Pour faire de l'apprentissage par transfert, on peut techniquement affiner n'importe quel modèle pré-existant. Un choix extrêmement commun comme point de départ est d'utiliser un modèle pré-entraîné sur le dataset ImageNet .

ImageNet est une vaste base de données d'images étiquetées. Elle contient des millions d'images classées en milliers de catégories, allant des objets du quotidien aux animaux. ImageNet a joué un rôle central dans les progrès du deep learning, notamment avec le défi ImageNet Large Scale Visual Recognition Challenge (ILSVRC), qui a permis le développement de réseaux neuronaux profonds.

En apprentissage par transfert, les modèles pré-entraînés sur ImageNet sont souvent utilisés comme point de départ, car ils ont déjà appris des caractéristiques visuelles générales à partir d'un large éventail d'images.

Q1. Étant données les différences marquées entre les caractéristiques des images d'ImageNet (des chats, des chiens, des autobus, des spatules, etc.) et les images OCT de la rétine, pensez-vous qu'utiliser un modèle pré-entraîné sur ImageNet va aider vos performances de classification? Expliquez votre raisonnement.

Q2. Affinez pendant au moins 10 époques un modèle pré-entraîné. Créez le en utilisant l'argument `pretrained=True`. Durant l'entraînement du modèle, faites une sauvegarde du modèle lorsque vous obtenez la meilleure *accuracy*. Affichez les courbes d'évolution de votre *loss* d'entraînement, et de votre *loss* et *accuracy* de validation.

Q3. Comparez les performances de votre modèle pré-entraîné avec celles de votre modèle non pré-entraîné. Qu'en concluez-vous?

Exercice 2.4: Évaluation finale

Jusqu'à présent on n'a évalué le modèle que sur le sous-ensemble de validation. Pour obtenir une évaluation finale de notre modèle, on doit l'évaluer sur l'ensemble de test.

Q1. Évaluez votre meilleur modèle sur l'ensemble de test. Affichez la *loss* et l'*accuracy* finale.

> Comme vous avez sauvegardé le meilleur modèle que vous avez obtenu à chaque expérience, ici serait l'occasion de charger le meilleur d'entre eux!

Q2. Commentez les performances de votre modèle sur l'ensemble de test et comparez-les à celles obtenues sur l'ensemble de validation.

Q3. Pourquoi est-il important de maintenir un ensemble de test séparé de l'ensemble de validation? Que se passerait-il si on n'avait que deux ensembles (entraînement et validation/test) plutôt que trois (entraînement, validation, test)?

Q4. Au sein de l'ensemble de test, identifiez

Une image CNV correctement identifiée comme CNV	Une image CNV incorrectement identifiée comme DME	Une image CNV incorrectement identifiée comme Drusen	Une image CNV incorrectement identifiée comme normale
Une image DME incorrectement identifiée comme CNV	Une image DME correctement identifiée comme DME	Une image DME incorrectement identifiée comme Drusen	Une image DME incorrectement identifiée comme normale
Une image Drusen incorrectement identifiée comme CNV	Une image Drusen incorrectement identifiée comme DME	Une image Drusen correctement identifiée comme Drusen	Une image Drusen incorrectement identifiée comme normale
Une image normale incorrectement identifiée comme CNV	Une image normale incorrectement identifiée comme DME	Une image normale incorrectement identifiée comme Drusen	Une image normale correctement identifiée comme normale

Affichez chaque image, son label, la prédiction de votre modèle, et l'index de l'exemple. Commentez vos observations.

Q5. Au vu des performances de votre meilleur modèle sur les données de test, seriez-vous à l'aise d'intégrer ce modèle dans des protocoles cliniques pour aider au diagnostic? Justifiez votre réponse.

Exercice 2.5: Visualisation de l'espace latent

Un problème de classification d'image peut essentiellement être vu comme la projection de vecteurs dans des espaces de hautes dimensions (e.g. les $h \times w$ pixels d'une image). La dernière couche d'un réseau de neurone doit ensuite apprendre à séparer ces vecteurs dans cet espace de hautes dimensions par des hyperplans. Ces vecteurs sont appelés *embeddings* ou "représentations latentes", car elles encodent l'information pertinente pour la tâche de classification. Notamment, le modèle qu'on utilise pour ce TP encode chaque image dans un espace de 960 dimensions. Cependant, visualiser ces espaces de hautes dimensions est impossible pour nous humains, car nous ne pouvons percevoir que

trois dimensions. Heureusement, il existe des techniques de réduction de dimensionnalité qui nous permettent de projeter ces espaces de hautes dimensions dans des espaces à deux ou trois dimensions, que nous pouvons visualiser. Parmi ces techniques, on retrouve la PCA (analyse en composantes principales) et le t-SNE.

Q1. Utilisez la fonction `get_embeddings` pour extraire les *vecteurs latents* des images de l'ensemble de test issus
1. d'un modèle aléatoirement initialisé;
2. d'un modèle pré-entraîné sur ImageNet, mais non affiné;
3. de votre modèle entraîné (sans pré-entraînement);
4. de votre modèle affiné (avec pré-entraînement).

Q2. Pour chacun des ensembles de vecteurs latents obtenus à la question précédente, utilisez la méthode t-SNE pour réduire la dimensionnalité des vecteurs à 2D. Affichez les résultats dans un graphique de dispersion (*scatter plot*), où chaque point représente une image, et où la couleur du point indique la classe de l'image. Commentez vos observations.

> Pour utiliser t-SNE, vous pouvez utiliser la classe `TSNE` de la librairie `sklearn.manifold`. Voici un exemple d'utilisation:
>
> ```
> from sklearn.manifold import TSNE
>
> tsne = TSNE(n_components=2)
> tsne_features = tsne.fit_transform(embeddings)
> ```
>
> Dans cet exemple, le paramètre `n_components=2` indique que nous voulons réduire les vecteurs à deux dimensions.

Références

[1] D. S. Kermany *et al.*, "Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning," *Cell*, vol. 172, no. 5, pp. 1122–1131, 2018, doi: 10.1016/j.cell.2018.02.010 .

[2] D. Kermany, K. Zhang, and M. Goldbaum, *Large Dataset of Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images*. (2018). Mendeley Data. doi: 10.17632/rscbjbr9sj.3 .